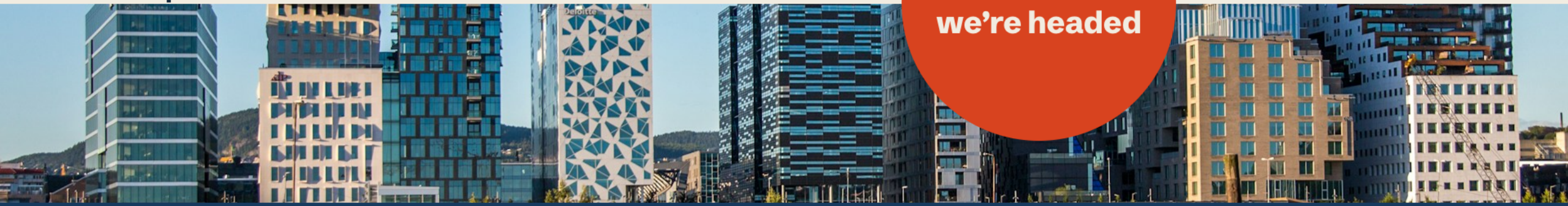
Anatomy of an AI Agent

- What is an agent
- Function calling
- Workflows are graphs
- Patterns
- The PR fixer
- Everything around the model
- Shipping it
- Managed agents

Where we're headed

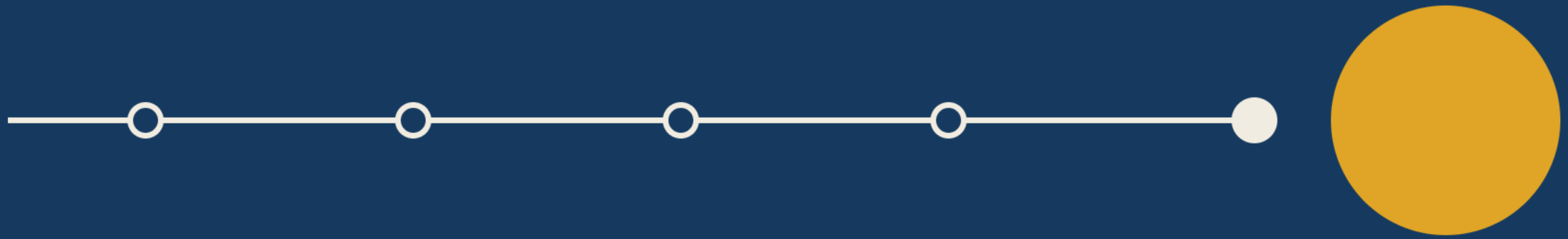
Tamas Piros, GDG Cloud Oslo, 2026
tamaspiros.com



Fresh Body 

Fit Mind 

Let's exercise! 



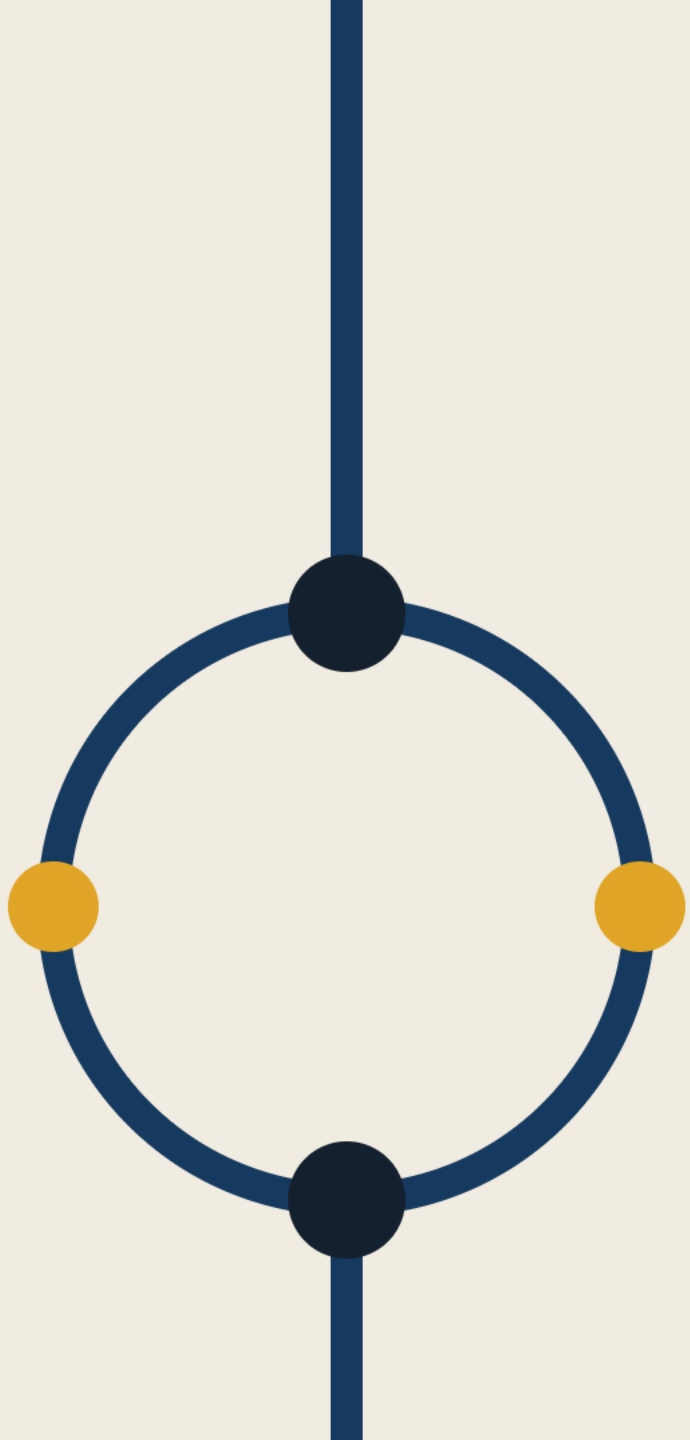
Thank you for the standing ovation!



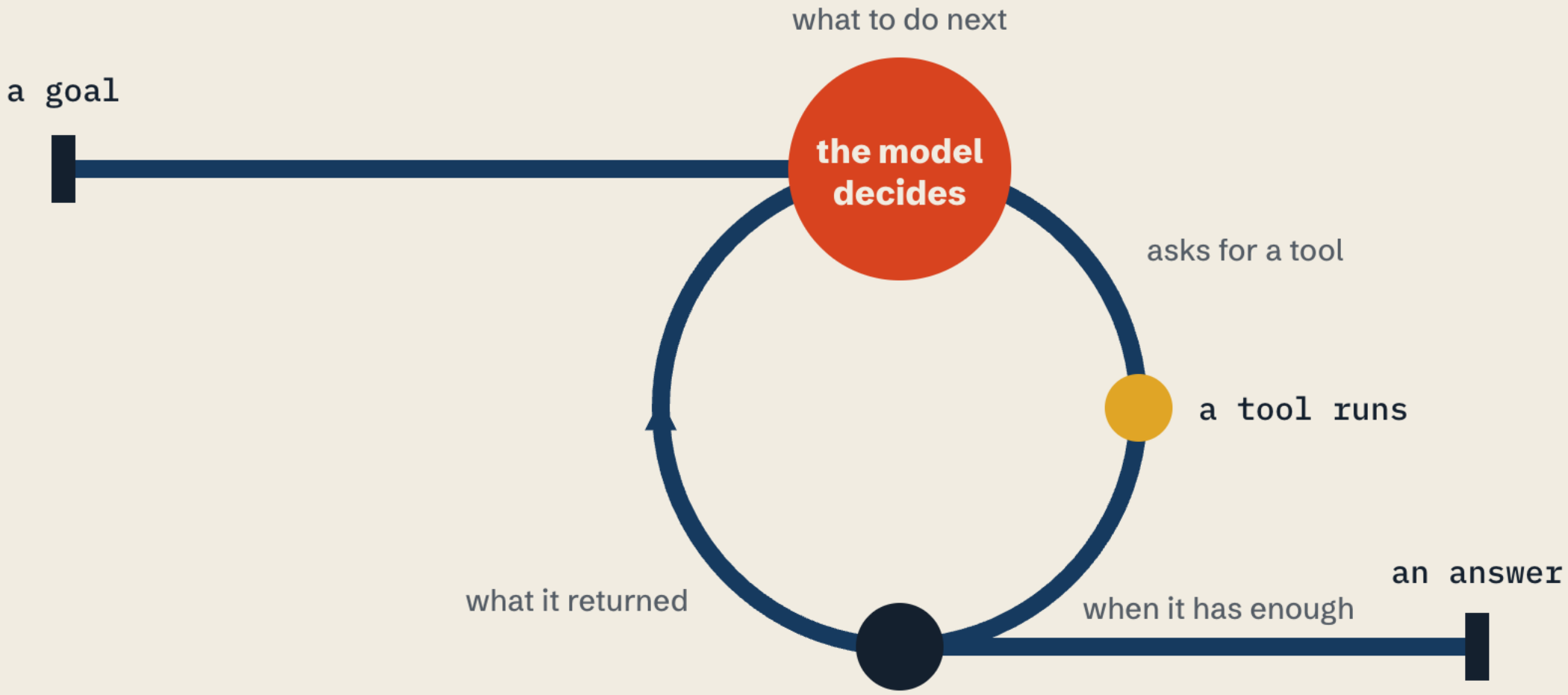
What is an agent?

A definition, and the loop hiding inside it.

A system where an LLM directs its own actions and tool use to pursue a goal over multiple steps, deciding what to do next from what came back.

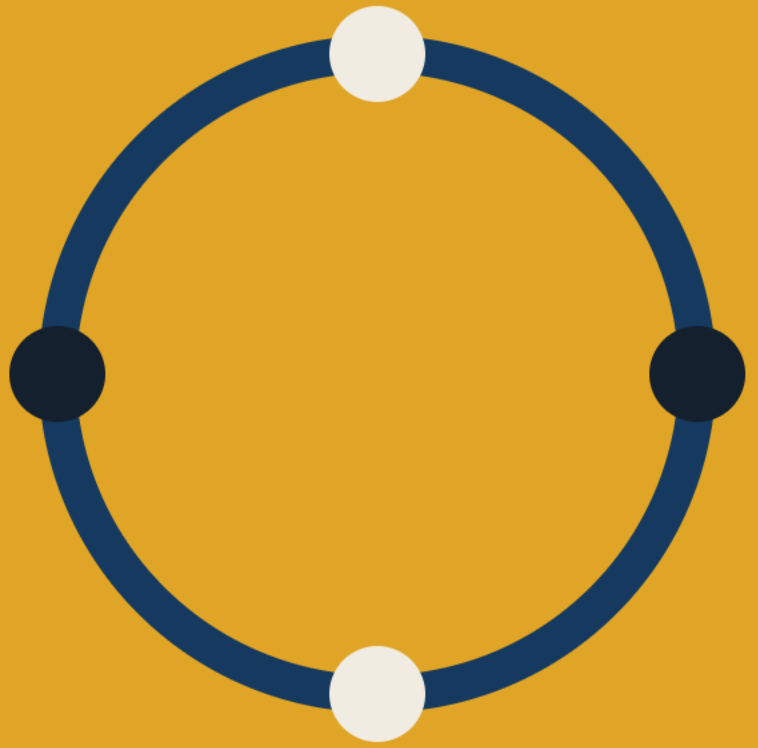


The model decides, your code acts



round and round until it is done. That loop is the agent.

A sophisticated loop.



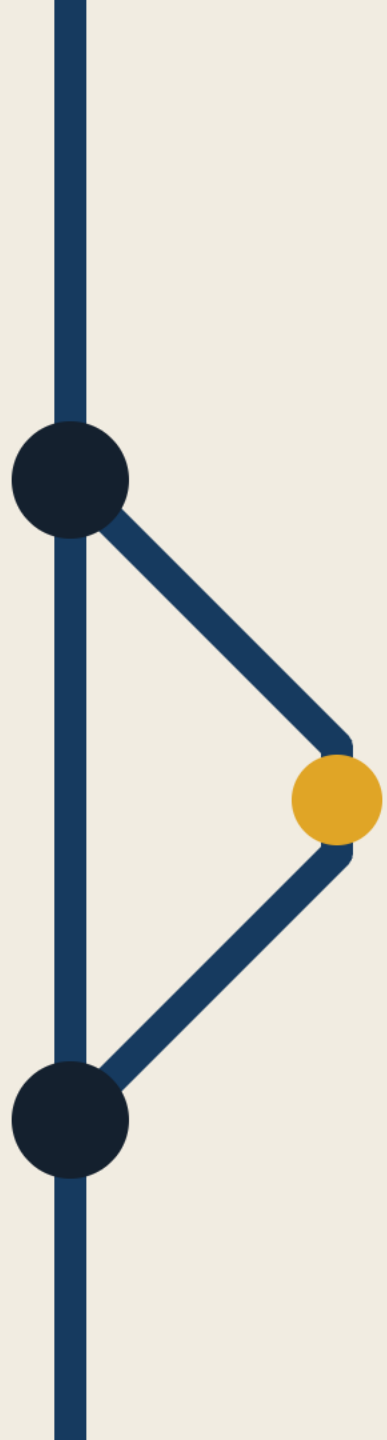
github.com/tpiros/gdg-cloud-oslo-2026



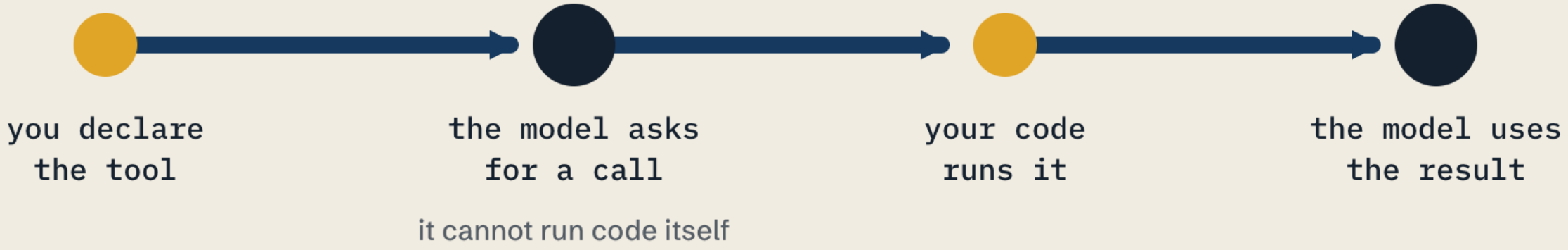
Function calling

How a model that cannot run code gets things done.

LLMs can request calls to functions you have exported to them, then use the results.



The model asks. Your code answers.



the model never touches your function. It only asks.

This is a FunctionTool




```
const swapiPeople = new FunctionTool({
  name: "swapi_people",
  description: "Search Star Wars characters by name.",
  parameters: z.object({
    name: z.string().describe("Character name to search for"),
  }),
  execute: async ({ name }) => { /* your code */ },
});
```

One line makes it ask permission

```
new FunctionTool({
  // …
  requireConfirmation: true,
  execute: async (a) => { … },
});
```

This is human in the loop: the run pauses, and a person approves or rejects the call before it runs.



Tools you did not write

```
new MCPToolset({
  type: "StdioConnectionParams",
  serverParams: {
    command: "node", args: [serverPath],
  },
});
```

- ADK asks the server for its tools at startup
- Each one becomes a tool the model can call
- Or connect to a remote server over HTTP

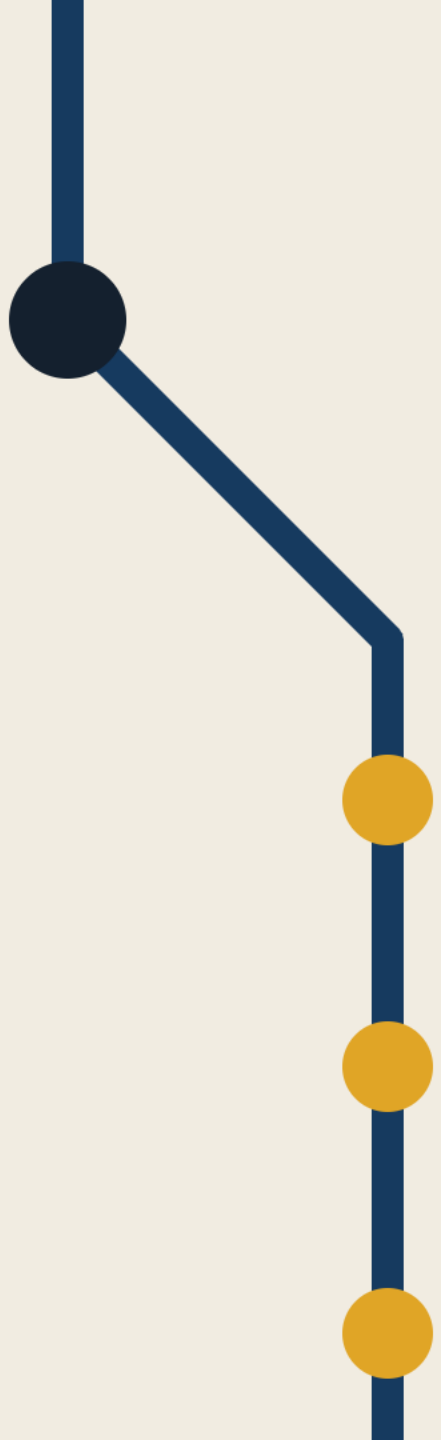


Workflows are graphs

How ADK 2.0 builds workflows, and why it means fewer model calls.

Workflows embed LLM calls in predefined code paths.

Your code controls the flow. The model fills in the steps.



Node, edge, graph



where the workflow begins

NODE: one step. An agent, a function, or a tool.

EDGE: an arrow. “When this finishes, run that, and pass the result.”

All the nodes and edges together = the GRAPH.

Each step is an LlmAgent when it needs a model, and a plain function when it does not.

new Workflow({ edges }) wires them into a graph. Each step's result is the next step's input.



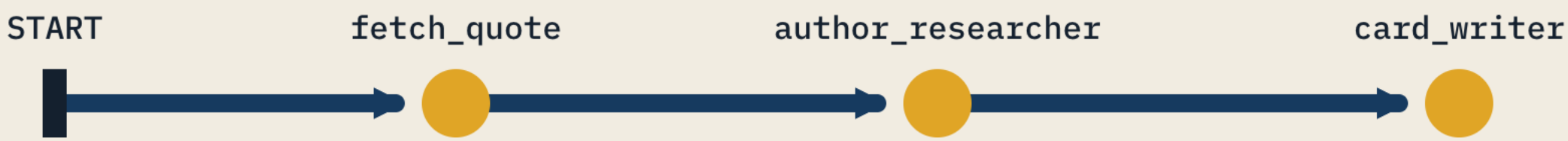
You list what connects to what



```
const rootAgent = new Workflow({
  name: "quote_pipeline",
  edges: [["START", fetchQuote, authorResearcher, cardWriter]],
});
```

Read it aloud: start, fetch the quote, research the author, write the card.

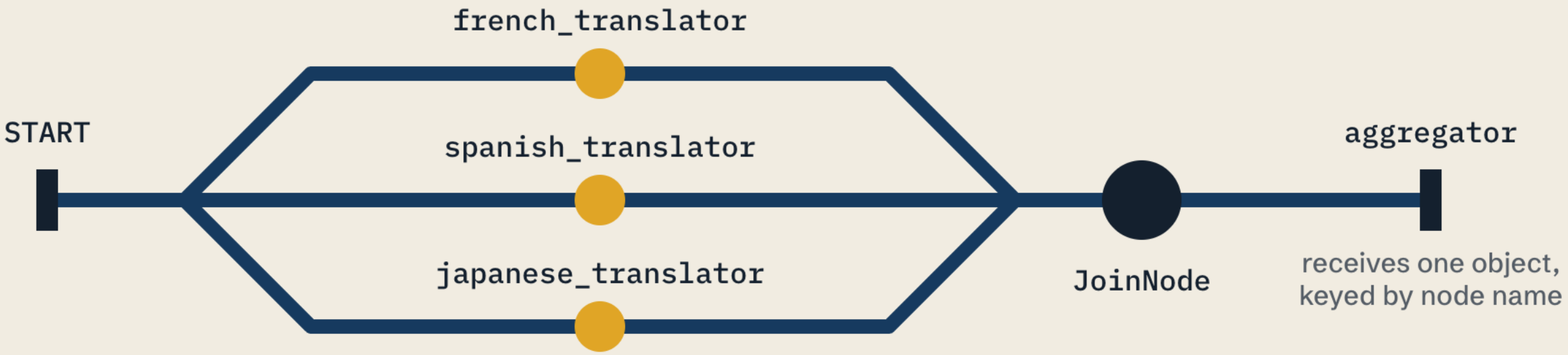
Sequential is a chain



edges: [["START", a, b, c]]

a, b, c: each an LlmAgent or a plain function. Workflow only wires them and passes each result on.

Parallel fans out, then joins

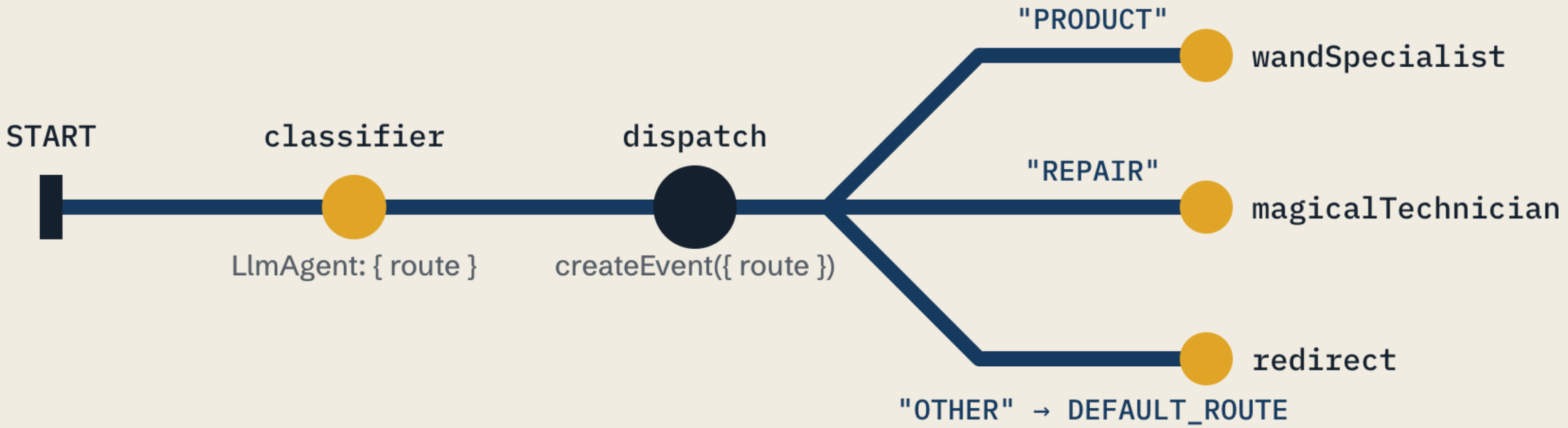


the join is a barrier: it waits for every branch to finish

branches only known at runtime? ParallelWorker runs one node over a list (agent-map.ts)

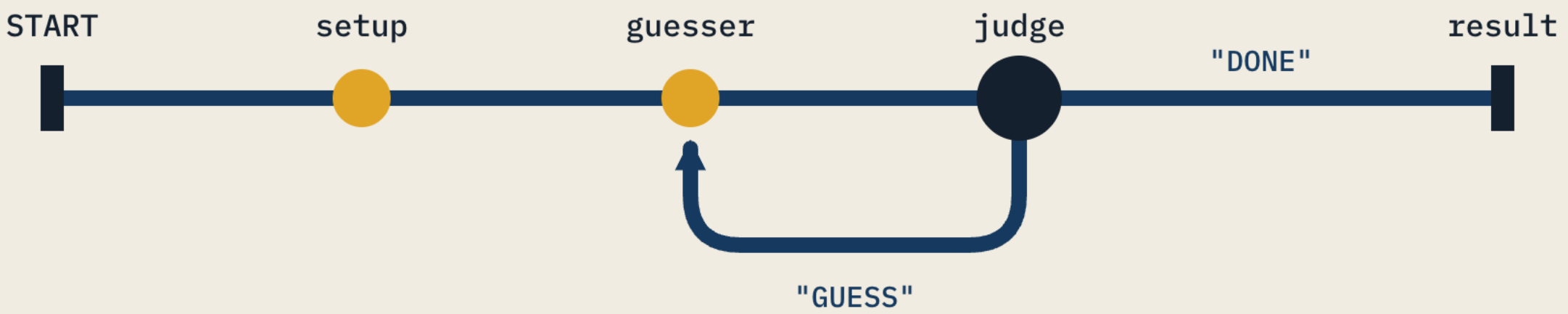
edges: [["START", [french, spanish, japanese], join, aggregator]]

A node picks the route




```
edges: [["START", classifier, dispatch],          // row 1: a chain, in order
        [dispatch, { PRODUCT: wandSpecialist,     // row 2: a fork. Take the
                     REPAIR: magicalTechnician,   // key == event.route, or
                     [DEFAULT_ROUTE]: redirect }]] // DEFAULT_ROUTE if none
```

A loop is an edge pointing backwards



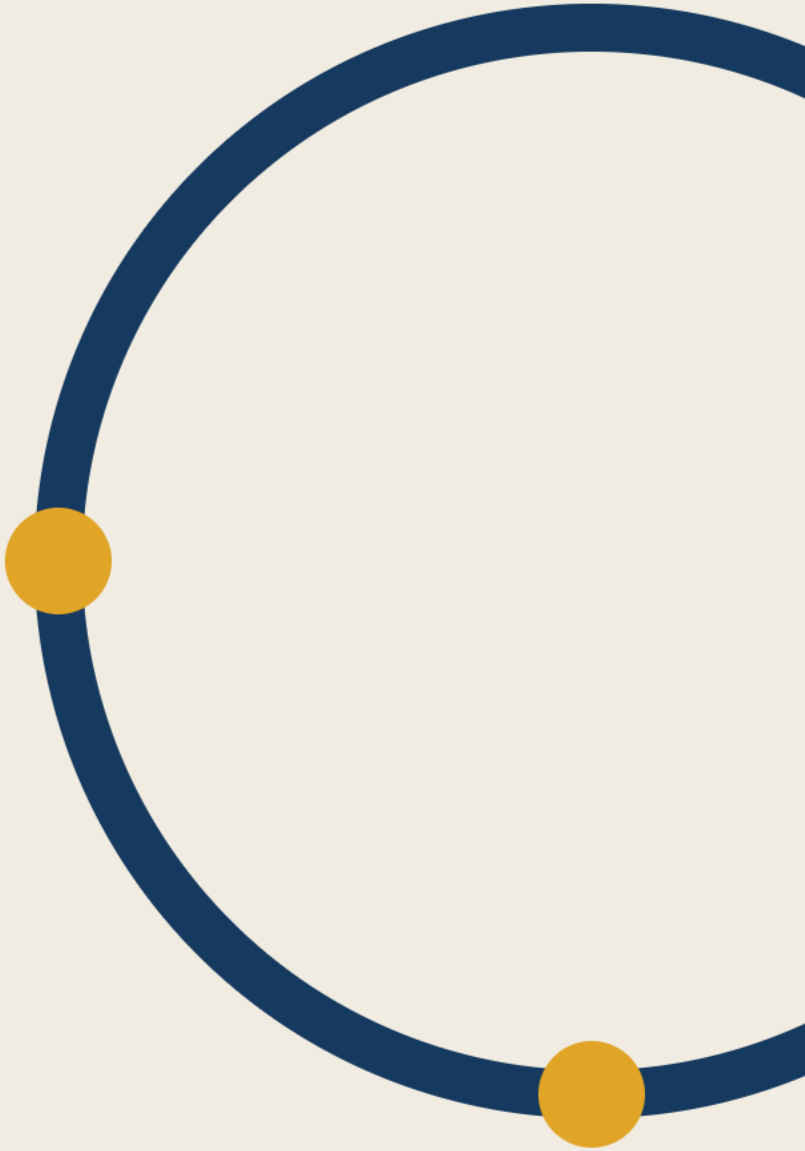
GUESS sends the run back to guesser, a node that has already run.

That backwards edge is the loop.

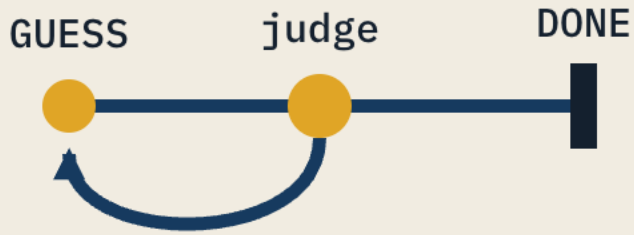
```
edges: [["START", setup, guesser, judge],
        [judge, { GUESS: guesser, DONE: result, [DEFAULT_ROUTE]: result }]]
        // judge is plain code that picks GUESS (go again) or DONE (stop)
```

An agent is a loop. The model decides when to stop.

A back-edge is a loop too. Your code decides when to stop.



A back-edge has no iteration cap. The node that picks the route has to guarantee the exit.




```
function judge(ctx: NodeContext, guess: Guess) {
  // a hit reveals a letter, a miss costs a life: every lap moves on
  const hit = !guessed.includes(letter) && secret.includes(letter);
  if (!hit) ctx.state.set("lives", lives - 1);
  const next = board(ctx);
  // two exits: no blanks left, or no lives left. LIVES is 6.
  const over = !next.pattern.includes("_") || next.livesLeft === 0;
  return createEvent({ route: over ? "DONE" : "GUESS", output: next });
}
```

If the loop calls a model, maxLlmCalls (default 500) is the only backstop.

Not every step needs a model



3 model calls per run

Only the steps that need judgement call a model. Fetching the quote is plain code and costs nothing.

Three ways to read a value in an instruction

FORM	READS FROM	USE IT WHEN
{author}	session state: whatever outputKey or ctx.state.set wrote	a value must outlive one step and be readable anywhere, later turns included
{Quote.author}	this node's input, i.e. the previous node's output. The label before the dot is ignored	the step right before you produced it (author_researcher reads fetch_quote's output)
<Quote.quote from fetch_quote>	the output of any earlier node in this turn, by node name	the value is two or more steps back and your input has moved on (card_writer's input is the bio, so the quote must come from fetch_quote)

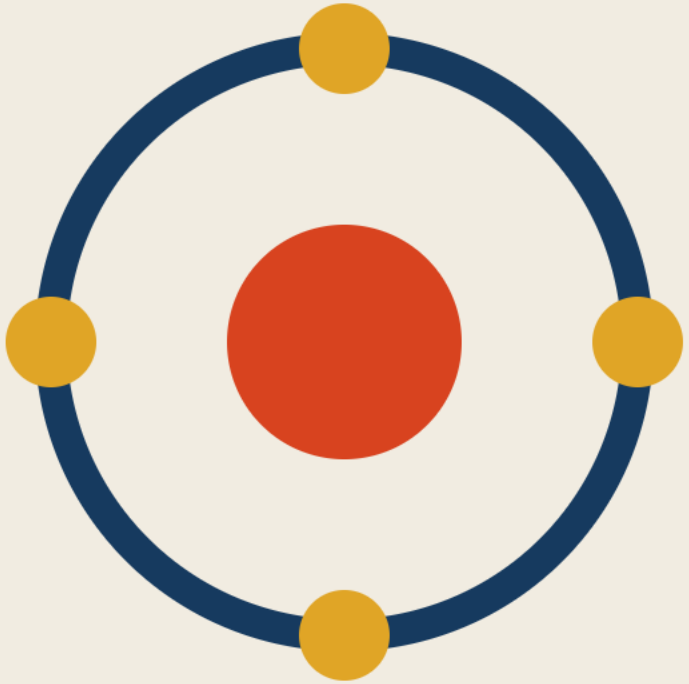
Add ? to make one optional: {author?} resolves to nothing instead of failing. All three are filled in before the model sees the prompt.



Patterns

When the model decides who works next.

What makes something a true agent is that the model controls the loop, including when to stop.



The model picks the next agent

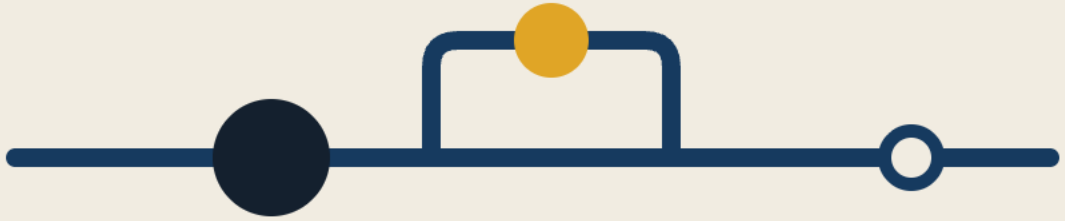
Coordinator

subAgents + transfer_to_agent. The model reads each sub-agent's description and hands the conversation over. Control moves to the specialist, and comes back only if it transfers back.



Agent-as-tool

tools: [new AgentTool({ agent })]. The specialist runs, its answer comes back as a tool result, and the coordinator replies. Control stays with the coordinator.



In a Workflow your code decides the next step. Here the model decides. Both are real ADK patterns, and the repo has both.

A whole workflow as one tool

```
const consultation = new Workflow({
  name: "wand_consultation",
  inputSchema: Question,
  edges: [["START", wandSpecialist, polish]],
});

const coordinator = new LlmAgent({
  name: "coordinator",
  model: "gemini-3.8-flash",
  tools: [new NodeTool(consultation)],
});
```

FunctionTool wraps a function. AgentTool wraps an agent. NodeTool wraps any node, even a whole graph.

The coordinator sees one tool. Its parameters come from the workflow's inputSchema, its result is the graph's output.

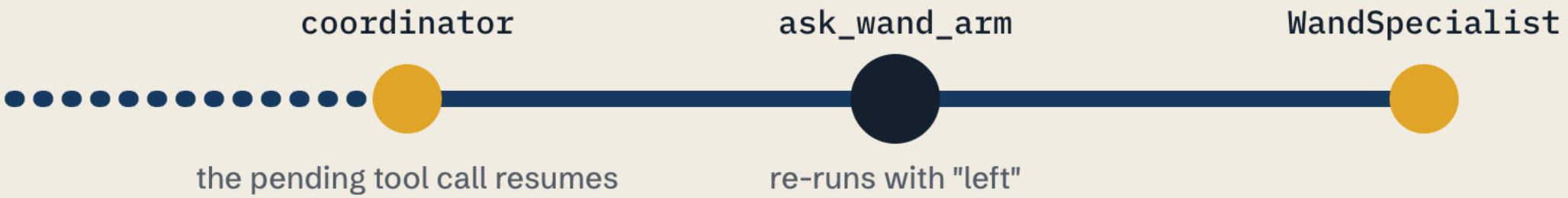
The model decides when to call it. The graph decides what happens inside. Every inner step shows up in the trace.

A node can pause the run and ask you a question

Turn 1



Turn 2: you reply “left”



Your next message is the answer. rerunOnResume: true runs the paused node again with it, and the graph carries on. Finished nodes do not re-run.

Same trick the PR fixer uses to ask before opening a pull request.



Seven patterns, all in the repo

PATTERN	HOW	DEMO
Coordinator and dispatcher	transfer_to_agent or AgentTool	routing-agent · agent-as-tool
Sequential pipeline	a Workflow chain	sequential-agent
Parallel fan-out and gather	JoinNode or ParallelWorker	parallel-agent
Hierarchical decomposition	NodeTool: a workflow as a tool	agent-as-tool (nodetool)
Generate and review	a plain checking node after the model	pr-fixer-agent
Iterative refinement	a back-edge with a guaranteed exit	loop-agent
Human in the loop	requireConfirmation · RequestInput	single-agent · pr-fixer-agent

Next: one agent that uses most of these at once.



The PR fixer

Everything so far, on a real task: a GitHub issue in, a pull request out.

Type fix issue #1.

The agent clones the repo, reads the issue, writes the fix, reruns the tests and shows you the diff. Then it waits. Say yes and it opens the pull request.



Six steps in a row, then a yes or no

```
const intake = new LlmAgent({ … });          // model call 1
const fixer = new LlmAgent({ …, tools });     // model call 2
export const rootAgent = new Workflow({
  name: "pr_fixer",
  edges: [["START", intake, prepare, fixer, verify, review],
    [review, { OPEN: openPr, SKIP: skip, [DEFAULT_ROUTE]: skip }]],
});
// tools: list_files · read_file · write_file · run_tests, sandboxed
// prepare, verify, review, openPr, skip: plain functions, no model
```

Read it aloud: two LlmAgents, two model calls. The other five nodes are plain functions.

The run stops and asks before it pushes

Turn 1: “fix issue #1”



Turn 2: you reply “yes”



rerunOnResume: true runs review again with your answer. No retry edge: say no and the sandbox is thrown away.

The graph checks the model's work

Don't take the model's word for it. After the fix, verify runs the test suite again itself.

The repo starts out with failing tests on purpose. So the question is not 'is everything green?' but 'did anything that used to pass break, and did something get fixed?'

That is what lands on your screen: a test score like Tests: 2 fixed, 0 broke, the diff, and a yes-or-no. The model's own summary is just the cover letter.



Everything around the model

Prompt, context, harness. Where the real work went.

You can call Gemini in five lines.

So what is ADK for?



One tool call with @google/genai

```
const ai = new GoogleGenAI({});
const decl = { name: "swapi_people", parameters: { … } };
let response = await ai.models.generateContent({
  model: "gemini-3.8-flash", contents: "Who is Luke?",
  config: { tools: [{ functionDeclarations: [decl] }] } });
const call = response.functionCalls?.[0];        // the model asks
const result = await swapiPeople(call.args);      // you run it
response = await ai.models.generateContent({ …,   // you send it back
  contents: [ …, { role: "user", parts: [{ functionResponse:
    { name: call.name, response: result } }] }] });
```

You wrote the loop: declare, ask, read the call, run it, send the result back, ask again.

The same, with ADK

```
const swapiPeople = new FunctionTool({
  name: "swapi_people",
  parameters: z.object({ name: z.string() }),
  execute: async ({ name }) => { /* your code */ },
});
export const rootAgent = new LlmAgent({
  name: "star_wars_lookup",
  model: "gemini-3.8-flash",
  tools: [swapiPeople],
});
```

ADK runs the loop: declares the tool, reads the call, runs execute, sends the result back, repeats.



Prompt engineering

what you say to the model, one call at a time

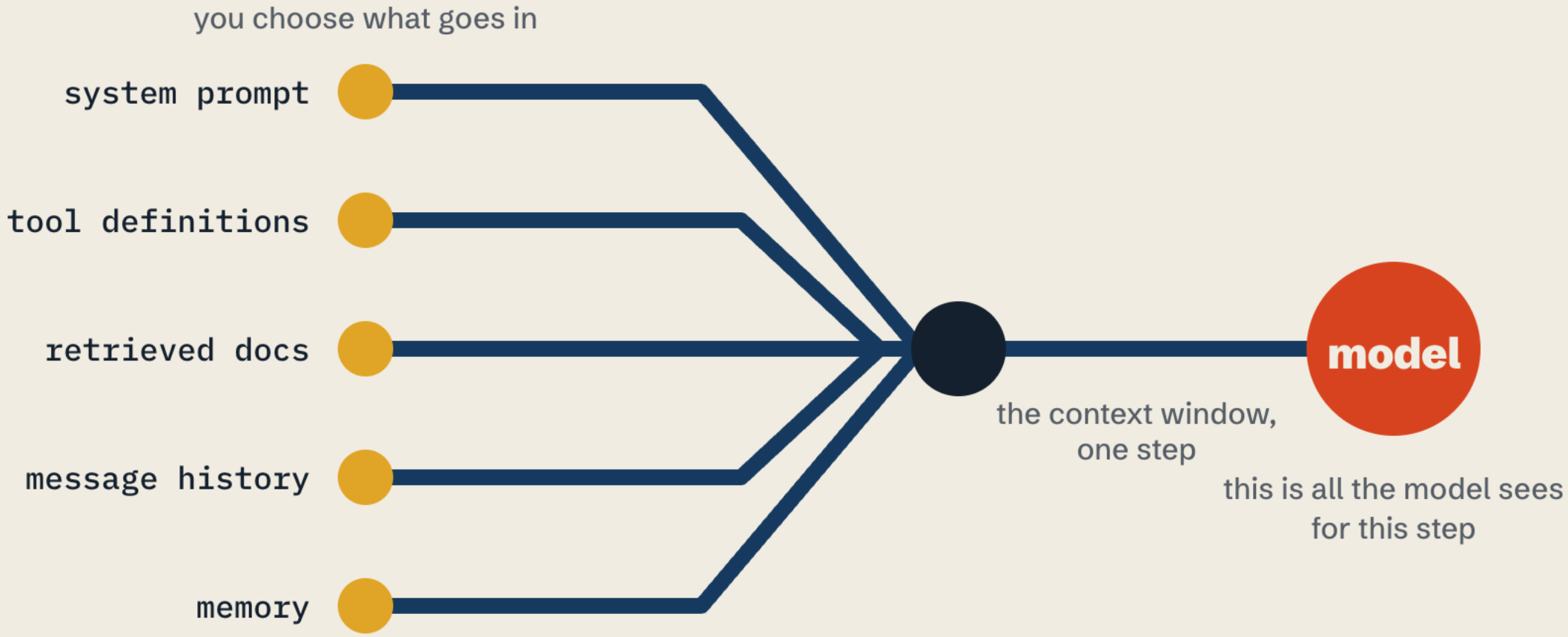
Context engineering

what the model gets to see at each step

Harness engineering

everything around the model: tools, graph, state, checks

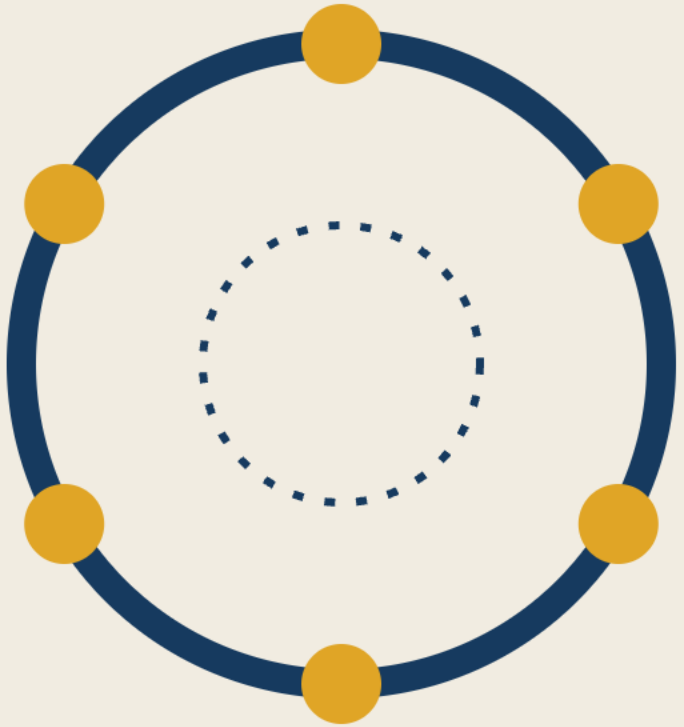
Deciding what goes into the window for a given step



context engineering is choosing these five things, every step

Harness engineering

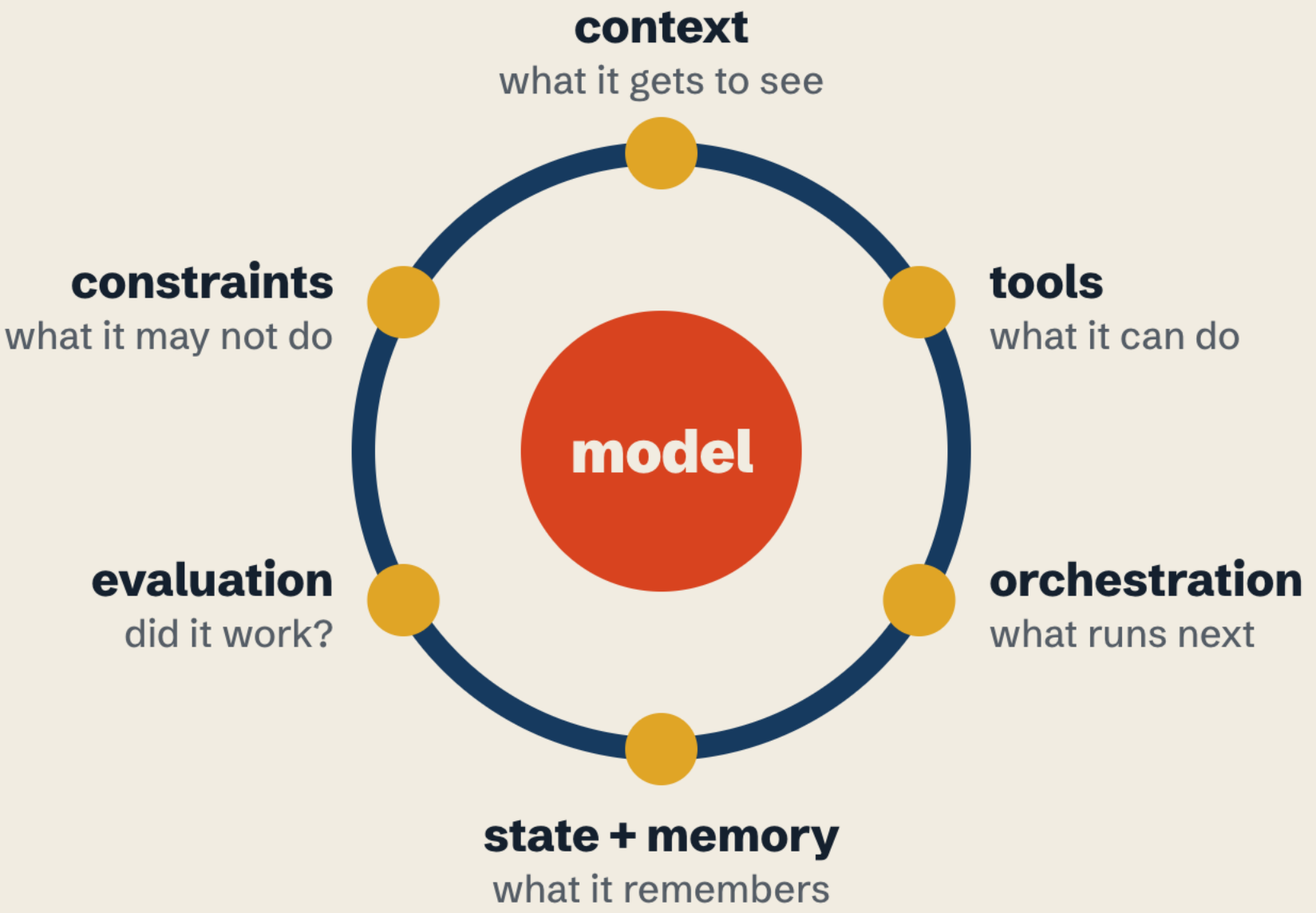
Everything in an agent that is not the model.



A model can't edit a file, run a test, or remember where it left off. The harness around it can.

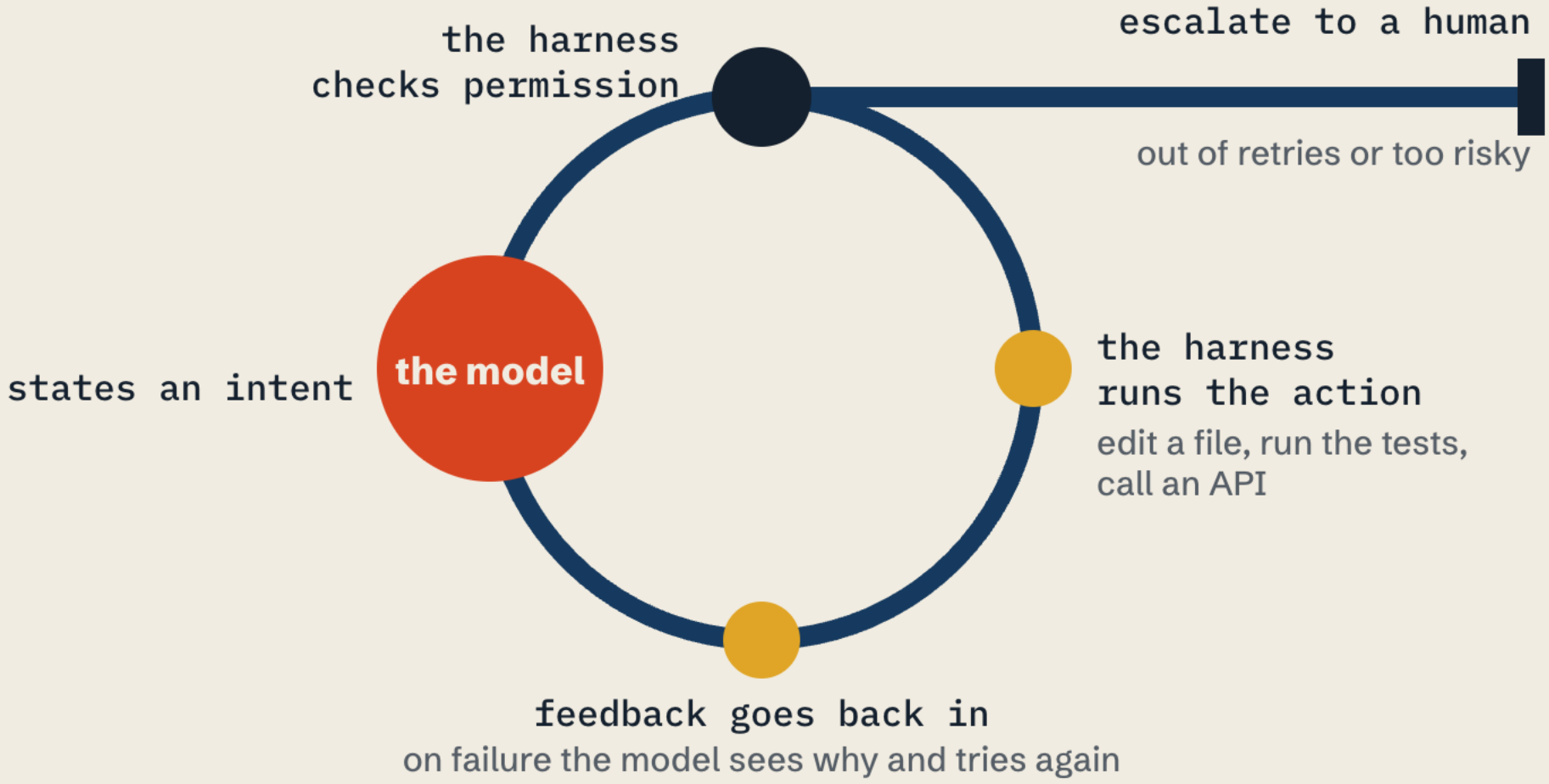


The harness



The model sits in the middle. Everything around it is code: yours, or the code ADK writes for you.

The core loop



The model only states an intent. The harness decides whether it runs, runs it, and feeds back what happened.

Pillar 1: context and memory



One conversation. Every message, tool call and result, in order, append-only.

SessionService

A key-value scratchpad. Structured values code reads directly, no tokens.

ctx.state, outputKey

Across sessions. Distilled facts found by meaning, not by key.

MemoryService

Three scopes, kept apart so the prompt doesn't rot. A value in state is read by plain code, and it costs no tokens.

Pillar 2: tools and execution

How the work gets done, and when it stops



Graphs, not prompt chains

code fixes the order, the model fills in the steps

MCP

tools you didn't write, discovered at startup, no glue code

Parallel checks

independent checks run side by side, a join waits for all of them

Retry ceilings

a fixed budget of attempts, then stop and hand it to a person

In the repo: sequential-agent · mcp-agent · parallel-agent · loop-agent. A back-edge has no cap of its own: the ceiling is a counter in your code, like the six lives in hangman.

Pillar 3: runtime security and privacy

Enforced in code, not requested in the prompt

In the repo: the PR fixer's four tools only touch its sandbox folder, and nothing is pushed until you say yes (pr-fixer-agent, single-agent).

Sandbox

a throwaway folder or container, never production

Untrusted data

repo code, uploads and web pages are data, never instructions

Approval gates

git push, database writes, payments: they wait for a human yes

Privacy

redact before it reaches the model, the traces or long-term memory

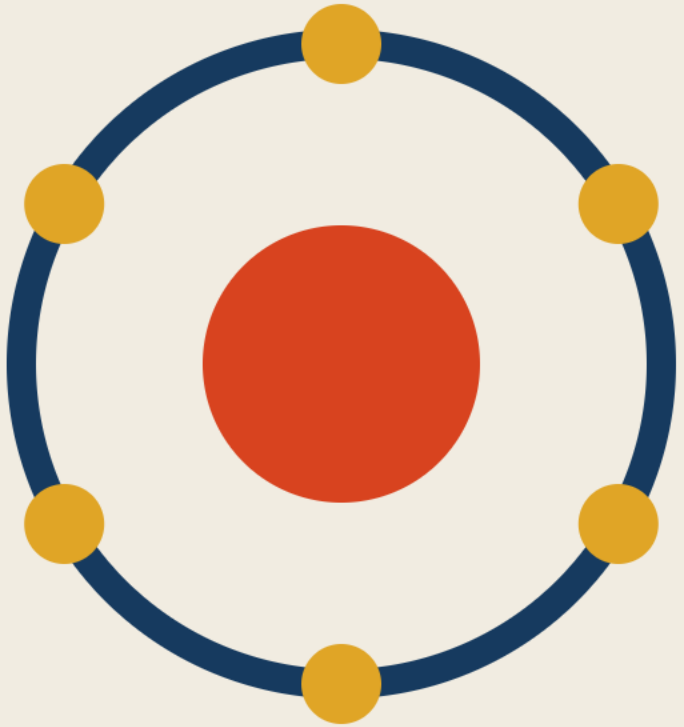
Pillar 4: evaluation

Three ways to check the work

TIER	HOW	WHAT IT CATCHES
Deterministic checks	tests, assertions, linters, schema validators	exact counts, valid formats
LLM as judge	a second model grades it against a rubric	clarity, accuracy, grounding
Human review	a developer or designer looks at real runs	usability, layout, odd edge cases

The PR fixer uses two of them: verify reruns the tests, and you read the diff before anything is pushed.

Every demo today ran on the same model. Everything that changed was the harness.





Shipping it

Cloud Run, or Google’s own runtime.

Deployment



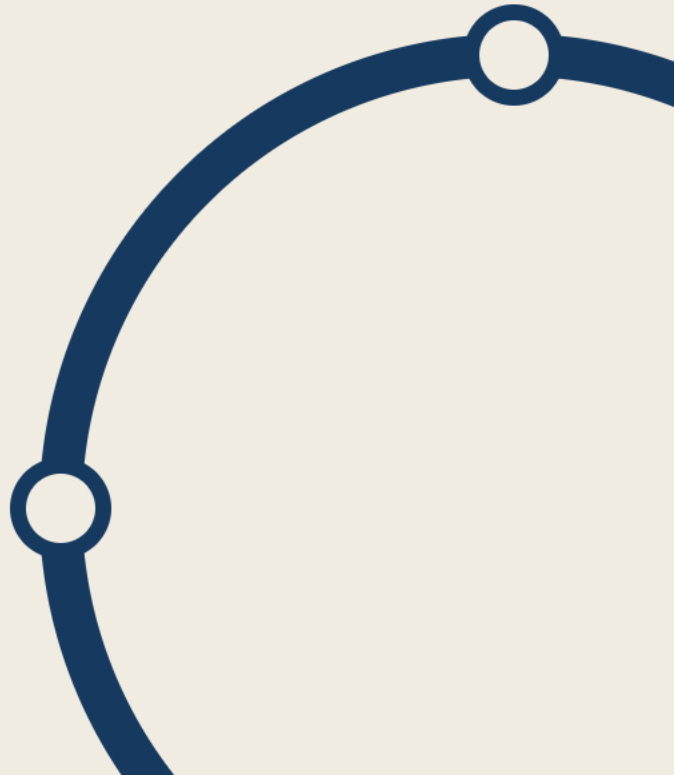
npx adk deploy cloud_run
npx adk deploy agent_engine



Managed agents

When you would rather not run the loop yourself.

Why would anyone hand the loop to someone else?



With ADK you write and run the loop: tools, graph, state, sandbox, retries.



A managed agent runs it for you. You send a message and read the result.

Why they exist

- A sandbox with a filesystem and a shell, created and thrown away per job
- Sessions that survive across turns, so turn two finds turn one's files
- Tools hosted next to the model, not on your laptop
- Long-running work, retries and traces without you babysitting a process

Google's version

Hosted agents such as the Antigravity agent, called through the Interactions API

environment: "remote" creates a sandbox on Google's side;
environment_id lets the next turn reuse it

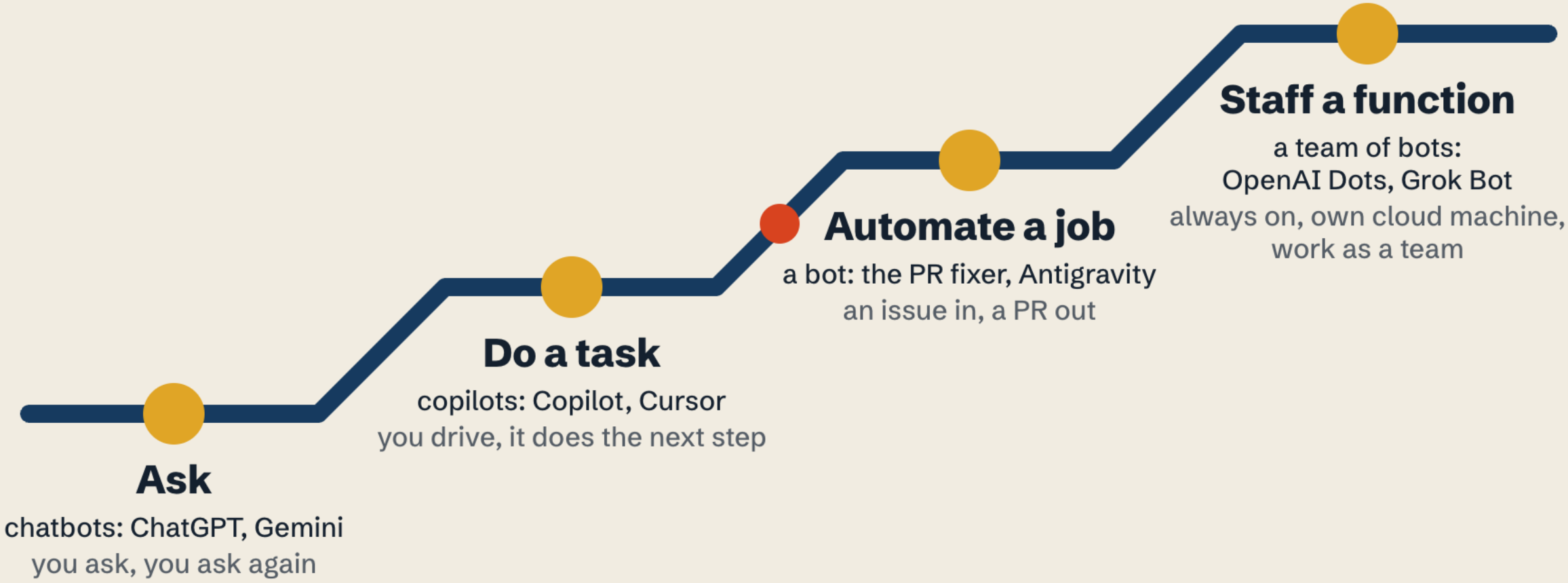
previous_interaction_id chains turns into one conversation

Directed by AGENTS.md and skills under .agents/skills/; no orchestration code on your side

Where we're headed

From asking a chatbot to staffing a function.

The maturity curve



Everything in this talk sits between the second and third dot. The fourth arrived in September 2026: always-on agents with their own machine in the cloud, working as a team.

Takk 🇳🇴

Tamas Piros

github.com/tpiros/gdg-cloud-oslo-2026
tamaspiros.com
linkedin.com/in/tpiros

